



**MUĞLA SITKI KOÇMAN UNIVERSITY
ELECTRICAL & ELECTRONICS ENGINEERING
THESIS TEMPLATE**

PID AND LQI CONTROL OF A BALL BALANCE TABLE

**Ahmet BARUT
Yusuf ÖZESER**

**Supervisor:
Asst. Prof. Dr. Fatma Yıldız TAŞCIKARAOĞLU**

ABSTRACT

PID AND LQI CONTROL OF A BALL BALANCE TABLE

Ahmet BARUT

Yusuf ÖZESER

B.Sc. Thesis, Electrical-Electronics Engineering Department

Supervisor: Asst. Prof. Dr. Fatma Yıldız TAŞCIKARAOĞLU

2025, (61) pages

In this study, we aim to design and implement two distinct controllers (PID and LQI) for a ball balancing table system and further improve the PID controller by using gradient descent algorithm. Controller systems are widely used in nearly all technological fields, especially in engineering applications. They are indispensable in industries such as automotive manufacturing, aerospace systems, high-speed rail technology, and many other advanced technological domains. In this study, both control methods were applied to the same physical system to show that the system can be controlled and that the methods can be used in practice.

Keywords: Ball balancing plate, PID tuning, LQI controller.

ÖZET

TOP DENGELEME MASASISININ PID VE LQI İLE KONTROLÜ

Ahmet BARUT

Yusuf ÖZESER

Lisans Bitirme Tezi, Elektrik-Elektronik Mühendisliği Bölümü

Tez Danışmanı: Dr. Öğr. Üyesi. Fatma Yıldız TAŞCIKARAOĞLU

2025, {61} sayfa

Bu çalışmada, bir bilyeli dengeleme masası sistemi için PID ve LQI olmak üzere iki farklı kontrolör tasarlanıp uygulanması ve PID kontrolörünün gradyan inişi algoritması ile optimize edilmesi amaçlanmaktadır. Kontrol sistemleri, özellikle mühendislik uygulamaları başta olmak üzere, hemen hemen tüm teknolojik alanlarda yaygın olarak kullanılmaktadır. Otomotiv üretimi, uzay-havacılık sistemleri, yüksek hızlı tren teknolojisi ve daha birçok ileri teknoloji alanında vazgeçilmez bir rol oynamaktadırlar. Bu çalışmada her iki kontrol yöntemi aynı fiziksel sistem üzerinde uygulanarak sistemin kontrol edilebilirliği ve uygulanabilirliği ortaya konulmuştur.

Anahtar Kelimeler: Bilye denge tahtası, PID optimizasyon, LQI kontrolcü.

ACKNOWLEDGEMENT

We owe our deepest gratitude to our advisor, Asst. Prof. Dr. Fatma Yıldız TAŞCIKARAOĞLU, for her guidance through this study, her constant ability to provide solutions to the problems encountered during the thesis, her encouragement and support both financially and morally in extracurricular scientific activities, and her positive approaches that accelerated the progress of this work. We are sincerely grateful for her patience, goodwill, and for generously sharing her knowledge and expertise. also wish to thank Ali Can ERÜST, who consistently offered his help whenever he saw me and never hesitated to assist when I asked. Even though he was not a participant in this thesis, his willingness to support me was greatly appreciated.

TABLE OF CONTENTS

ABSTRACT.....	i
ÖZET.....	iii
ACKNOWLEDGEMENT	v
LIST OF TABLES	ix
LIST OF FIGURES	xi
LIST OF ABBREVIATIONS	xiii
LIST OF SYMBOLS	xv
1. INTRODUCTION	1
1.1. Ball Balance Table	1
1.2. PID	1
1.3. LQI.....	2
2. MATERIAL AND METHODS	4
2.1. Ball Balance Table System	4
2.1.1 Components	4
2.1.2 System modeling.....	7
2.1.3 Equations of motion	9
2.1.4 Linearization the system	11
2.1.6 Modeling of actuator	12
2.1.7 Simplified block diagram of operation	12
2.2. PID Tuning by Gradient Descent Algorithm	13
2.2.1 Gradient descent.....	13
2.2.2 Implementation of gradient descent	14
3. RESULTS	15
3.1. Tuned PID Controller Simulation	15
3.1.1 Step responses with different step sizes	15
3.1.2 Cost progression with different step sizes.....	17

3.1.3 Effect of step size on iteration time.....	19
3.1.4 Final results on workspace and simulink	20
3.2. LQI	23
4.REFERENCES	31

LIST OF TABLES

Table 1. Ball Balancing Table Parameters	7
Table 2 Peak value, rise time and settling time at 5000 iteration for different learning rates	17
Table 3. Peak value, rise time and settling time at 20000 iteration for different learning rates	17
Table 4. X-axis cost at 5000 iterations for different learning rates.....	18
Table 5. X-axis cost at 20000 iterations for different learning rates.....	18
Table 6. Effect of Learning Rates on Iteration Time	19

LIST OF FIGURES

Figure 1. Working Principle of PID Controller	2
Figure 2: LQI block diagram.....	3
Figure 3. Ball Balance Table.....	4
Figure 4. RC Servomotor	5
Figure 5. Operation of Resistive Touchscreen.....	6
Figure 6. Physical Model of Ball Balance Table	7
Figure 7. Free body diagram	9
Figure 8. Relationship between motor angles and plate angles	10
Figure 9. Relationship between motor angles and plate angles	11
Figure 10. Control schematic of operation.....	12
Figure 11 Operation of Gradient Descent Algorithm	13
Figure 12.X-axis step responses of different step sizes at 5000 Iteration.....	16
Figure 13. X-axis step responses of different learning rates at 20000 iteration.....	16
Figure 14. X-axis cost vs. number of iterations for different learning rates	18
Figure 15. Step responses of X and Y-axis in 2D (Workspace)	20
Figure 16 Step response of system in 3D (Workspace)	20
Figure 17. Step response in Simulink	21
Figure 18. System's response to sinusoidal input in simulink.....	21
Figure 19. System's response to square input in simulink	22
Figure 20. System's response to sawtooth input in simulink	22
Figure 21: The block diagram of the system.....	23
Figure 22: The graph of the LQR states.....	25
Figure 23:The output of the x-axis, and the y-axis.	25
Figure 24: The block diagram of LQI.....	26

Figure 25: The output of LQI control with all states.....	29
Figure 26: The output of x-axis and y-axis of the LQI.	29
Figure 27: The new output with ignoring the performance.	30
Figure 28:The new output with ignoring the cost	30

LIST OF ABBREVIATIONS

LQR Linear-Quadratic-Regulator

LQI Linear-Quadratic-Integral

MIMO Multi Input Multi Output

LIST OF SYMBOLS

K_i	Gain of the integral
K_x	gain of States
Q	Cost weight of the state variable
R	Cost weight of the control signal
P	Riccati equation
K_K	Gain of the system
T_s	Sampling time
J	Cost function
$\tilde{A}_d^T$	Transpose of extended A matrix converted to discrete time.
$\tilde{B}_d^T$	Transpose of extended B matrix converted to discrete time.
$G_M(s)$	Motor transfer function.
$G_{BBT,x}(s)$	Transfer function in x-direction of the plant.
$G_{BBT,y}(s)$	Transfer function in y-direction of the plant.
m	Mass

1. INTRODUCTION

Control theory is a field of engineering and applied mathematics that deals with the behavior of dynamical systems. Its main goal is the design of controllers that influence the input of a system in order to produce a desired output in spite of disturbances or uncertainty. Through the use of feedback, systems can be stabilized, performance can be optimized, and specific goals such as setpoint tracking or disturbance rejection can be achieved. Control theory has wide range of use in modern world, such as in the fields of robotics, aerospace, automobile systems, and industrial automation.

1.1. Ball Balance Table

The ball balance table is a classic and widely studied experimental platform in control engineering. It consists of a ball that is free to move on a flat surface, where the surface's inclination is manipulated to control the ball's position. The system is naturally unstable, meaning that without feedback control, the ball will quickly roll off the table.

Because of its nonlinear and multi-variable dynamics, the ball balance table serves as an ideal benchmark for testing and comparing control algorithms. It is commonly used to demonstrate the effectiveness of various controllers and techniques. Numerous studies have utilized this system to illustrate the applicability of different control methodologies. Wang, Li and Zhao's study (2012) demonstrated that neural networks are effective for nonlinear system identification, with NARMA-L2. Nunez, Acosta, Jiménez (2017) provided insights into real-time tracking and statefeedback control for ball-and-plate systems.

1.2. PID

Proportional integral differential control (PID control) is a control method widely used in industrial process control, which can adjust the output of the control system, control the object through the output of the control system, and reduce the error between the controlled object and the set value (or input). The PID controller is structured around 3 main parameters. These are: a proportional unit K_p , an integral unit K_i and a derivative unit K_d . The function of the proportional unit is to adjust the proportion of the control signal according to the magnitude of the deviation, and quickly reduce the deviation. The function of the integration unit is to gradually reduce the output value until the set value is reached by controlling the integration effect of the signal

according to the accumulation of error. The role of the differential unit is to predict error trends and adjust control signals to reduce errors, thus achieving effective control in systems where accurate mathematical models are difficult to build. Working principle of a PID controller is shown in the diagram below.

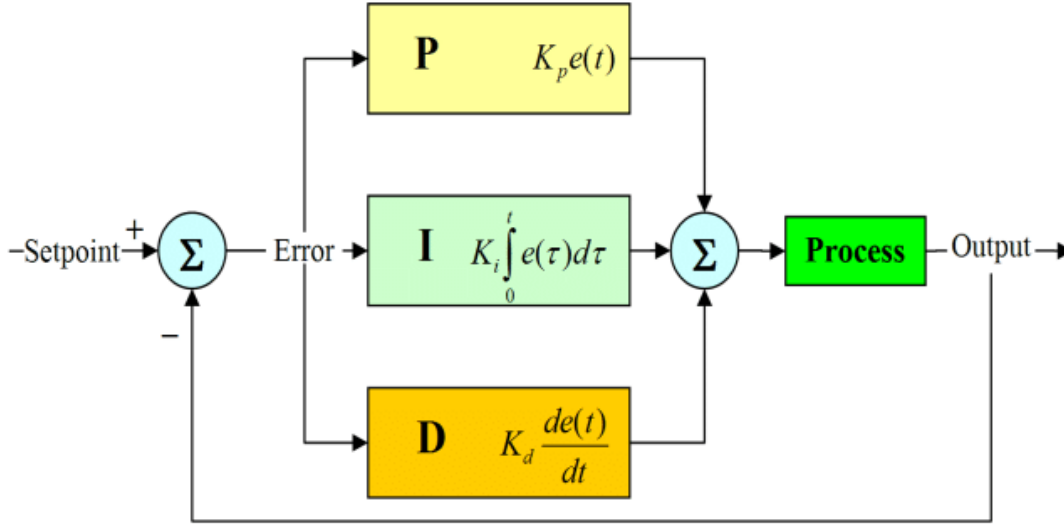


Figure 1. Working Principle of PID Controller

In practice, the parameters of the PID controller can be adjusted online or according to actual work experience to achieve the desired control requirements. (Haowen Jiang 2024). In this paper, these parameters will be adjusted (or tuned) by using gradient descent method, which is a crucial topic in optimization.

1.3. LQI

The Linear Quadratic Integral (LQI) is another controller type which has a widely used area as PID controller. First, Rudolf Kalman introduced the Linear quadratic Regulator, Linear Quadratic Regulator (LQR) is a widely applied optimal control technique in modern control systems (Abraham, 2024). The LQR can control the linear system with more than one input and output, and it is called Multi Input Multi Output (MIMO) (Parlar, 2021). LQR aims to set the system state variables to zero, so we cannot send any references to the controller, however LQI is an extension of the Linear Quadratic Regulator (LQR) method. While LQR aims to drive the state of the linear system to zero, the LQI method incorporates both state feedback and integral action to eliminate the steady-state error.

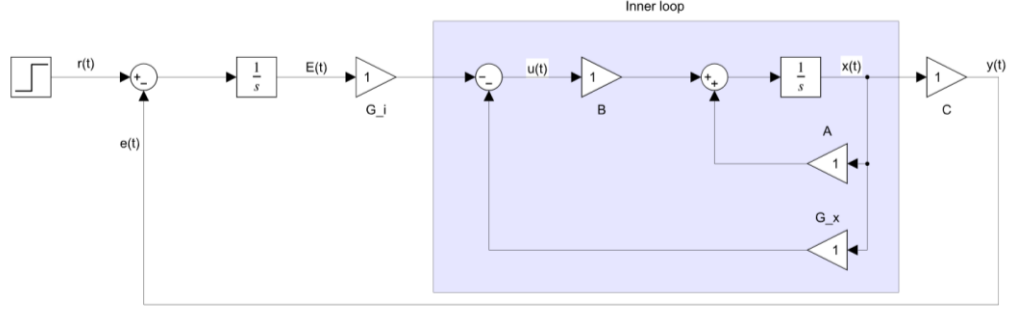


Figure 2: LQI block diagram

The state space matrix:

$$\dot{x} = Ax(t) + Bu(t) \quad (1)$$

$$y = Cx(t) + Du(t)$$

Generated the system matrix according to the plant. Then the important thing is that to obtain the gains of system. The gain defined by the weight of the Q and the R. LQI controller can be defined as: it operates to minimize the cost and increase performance. The Q represents the weight of the cost of system's states, and the R is the weight cost of the control input.

The cost function for continuous time.

$$J = \int_0^{\infty} \left[(\tilde{x}^T(t) \tilde{\varepsilon}^T(t)) * Q * (\tilde{x}^T(t) \tilde{\varepsilon}^T(t))^T + \tilde{u}^T(t) * R * \tilde{u}(t) \right] dt \quad (1.1)$$

The matrix Q primarily affects the performance of the system, while the matrix R influences the amount of energy control utilized by the system.

The optimal feedback controller.

$$u(t) = -[K_{state}, K_{Integrate}] \begin{bmatrix} x(t) \\ \varepsilon(t) \end{bmatrix} \quad (1.2)$$

Finally, the integrator accumulates the error over time and generates a signal to eliminate the steady-state error. Hence, we are able to send a reference to the LQI controller compared to LQR controller.

2. MATERIAL AND METHODS

Every thesis which will carry the name of Muğla Sıtkı Koçman University, Engineering Faculty must meet the same high standards of presentation in terms of all pertinent physical properties, including format as well as paper and print quality. The following guidelines are related to the format and appearance of the thesis that you are going to prepare.

2.1. Ball Balance Table System

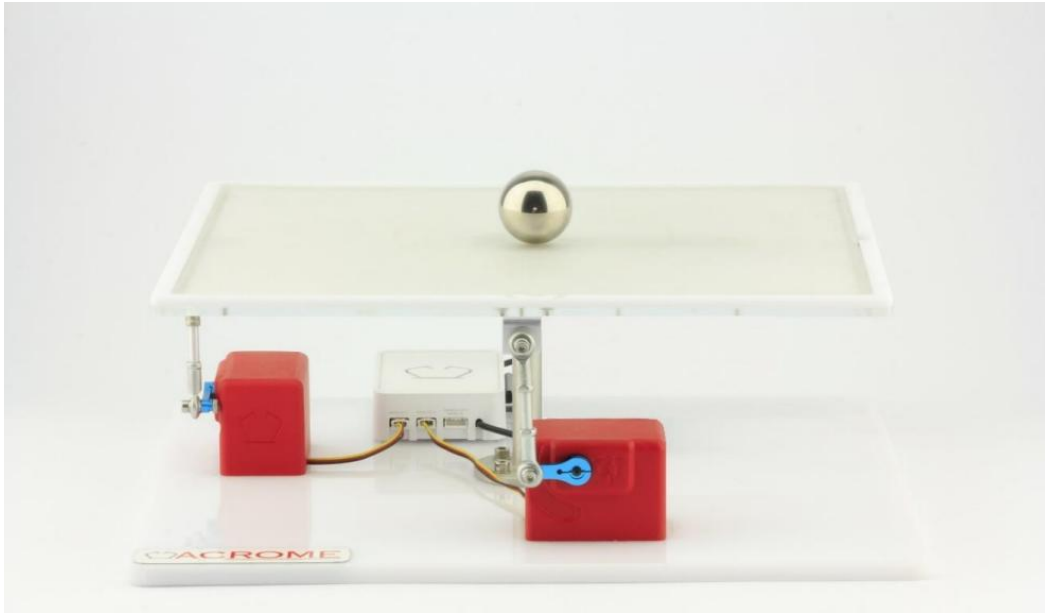


Figure 3. Ball Balance Table

2.1.1 Components

2.1.1.1 RC servomotor

RC servos are electromechanical devices that convert electrical signals to rotary movement. They provide simple and handy solutions to most of control and robotic applications. RC servomotors consist of several parts that include controller, potentiometer, gearbox, driveshaft and motor as shown below.

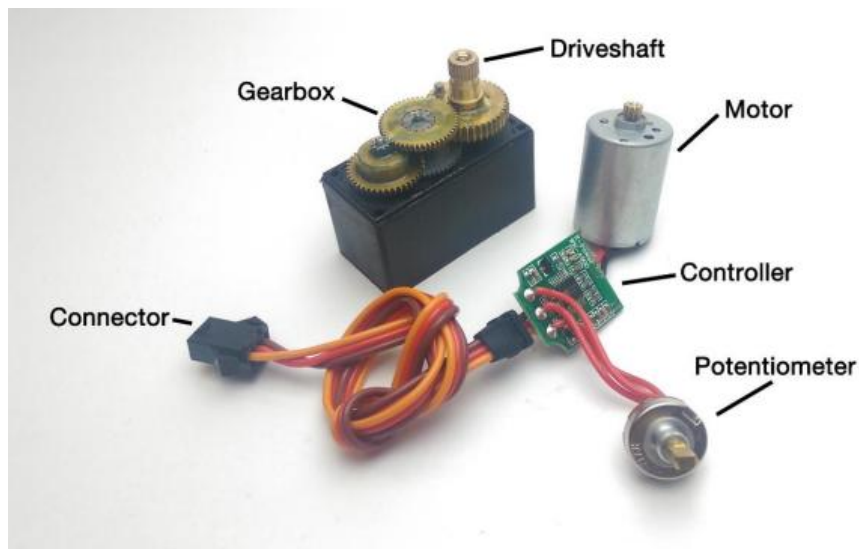


Figure 4. RC Servomotor

2.1.1.1.1 Controller

Circuit that is responsible of reading the control signals (PWM signals –more about PWM signals can be found in Chapter 2) and controlling the motor accordingly. Controller circuits determine the type of servos; they can be either digital or analog. Analog servos handle PWM signals up to 50 Hz frequency. Digital servos handle these PWM signals more precisely. They can decode signals up to 330 Hz. This difference in decoding can provide more torque to the motors. In addition to that; digital servos can be programmed to change direction, dead band width etc. Generally; digital servos are more advantageous than analog servos in expense of cost and power consumption.

2.1.1.1.2 Potentiometer

Position feedback of the main shaft is provided by the potentiometer. It is attached to the driveshaft so the rotation of the driveshaft results in different resistances on the potentiometer. By reading the resistance values, controller knows exact driveshaft angle.

2.1.1.1.3 Motor

The motor of the servo is high speed DC motors which are controlled by H-bridges located inside their controller circuitry.

2.1.1.1.4 Gearbox

The gear set is located between the driveshaft and the motor. It regulates the RPM of the motor resulting in lower movement speed and more torque.

2.1.1.1.5 Driveshaft

Driveshaft is the output of the whole system. It is the actual component that rotates to the desired angle.

2.1.1.2 Arduino Mega 2560

Arduino is a portable embedded device that allows users to design and control robotic or mechatronic systems.

2.1.1.3 2D Resistive Touch Screen and Controller

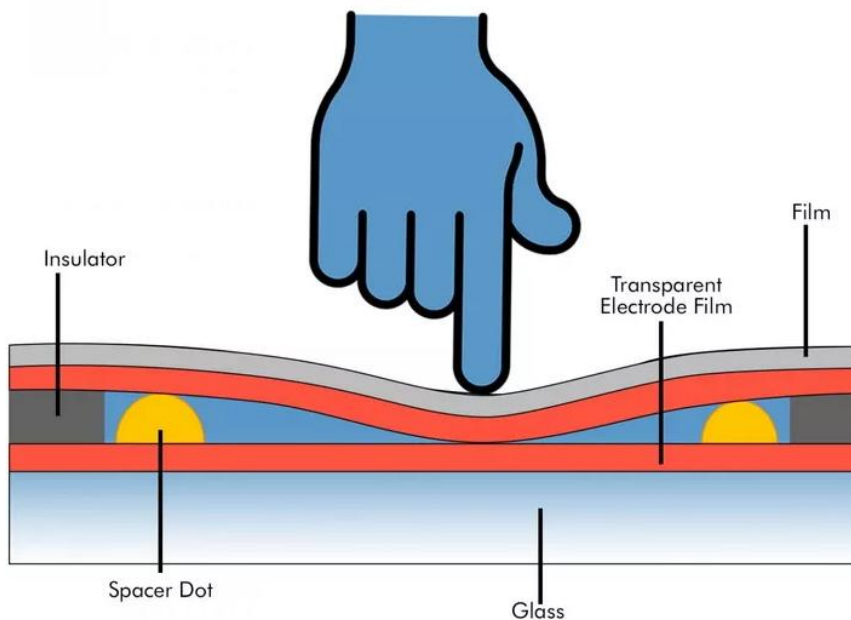


Figure 5. Operation of Resistive Touchscreen

The position of the ball on the table is measured using a 17-inch, 4-wire resistive touchscreen. This type of touchscreen consists of two conductive layers separated by an air gap. A varying voltage is applied across each layer using resistive components. When pressure is applied—such as by the ball—the layers come into contact, and the resulting voltage values are measured by a touchscreen controller. These voltage readings are then converted into screen coordinates, representing the ball's position.

The operation of it is illustrated in figure 4. The touchscreen used in the ball-balancing table is capable of providing position data at a rate of up to 100 Hz through a digital controller, allowing for smooth and responsive feedback acquisition.

2.1.1.4 Power Distribution Box

Two RC servomotors and Arduino connections are located on power distribution box.

2.1.2 System modeling

The physical model of the Ball Balancing Table is shown in Figure 5. The model describes the kinematic system of the Ball Balancing Table. According to the physical model, the actuators of the system (motor x and motor y) are attached rigidly to the base plate of the system. The actuators are connected to the table over links with ball joints on both sides. Table is connected to the base plate of the system over a rotary joint with two degree of freedom. The physical model also illustrates the linkage parameters of the table which are necessary for the mathematical model of the system. These parameters are also explained table 1.

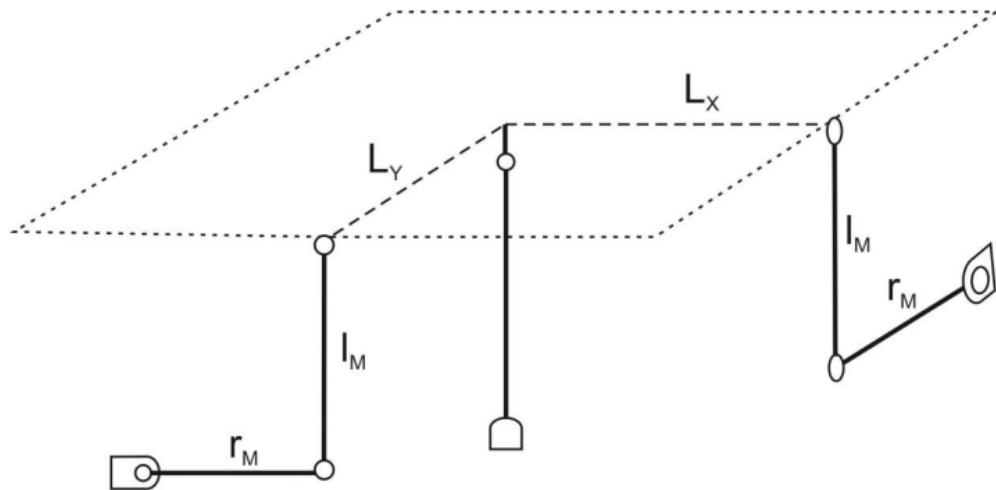


Figure 6. Physical Model of Ball Balance Table

Table 1. Ball Balancing Table Parameters

Symbols	Definiton	Value	Unit
L_x	Beam Length in x-direction	0.134	[m]
L_y	Beam Length in y-direction	0.168	[m]

r_M	Motor Arm Length	0.0245	[m]
r_b	Radius of the Ball	0.02	[m]
m_b	Mass of the Ball	0.26	[kg]
J_b	Rotational Inertia of the Ball	0.0000416	[kg*m2]
g	Acceleration due to Gravity	9.81	[m/s2]
α	Plate Angle about x-axis	(Variable)	[degrees]
β	Plate Angle about y-axis	(Variable)	[degrees]
θ_x	Motor Angle for x-axis	(Variable)	[degrees]
θ_y	Motor Angle for y-axis	(Variable)	[degrees]

In order to achieve the equations of motion for a dynamical system consisting of a ball on a platform the following simplifications has to be assumed:

- The ball is rolling and not slipping on the platform
- No friction is considered
- The geometry of the ball is perfectly spherical and homogeneous
- The ball has no translation upwards relative to the platform

The first step of modeling is to derive the equation of motion. The equation of motion describes the relationship between motion of the ball (x, y) and the two angles of the table (α , β). In the next section, the equation of motion will be derived with newtoniaon n method.

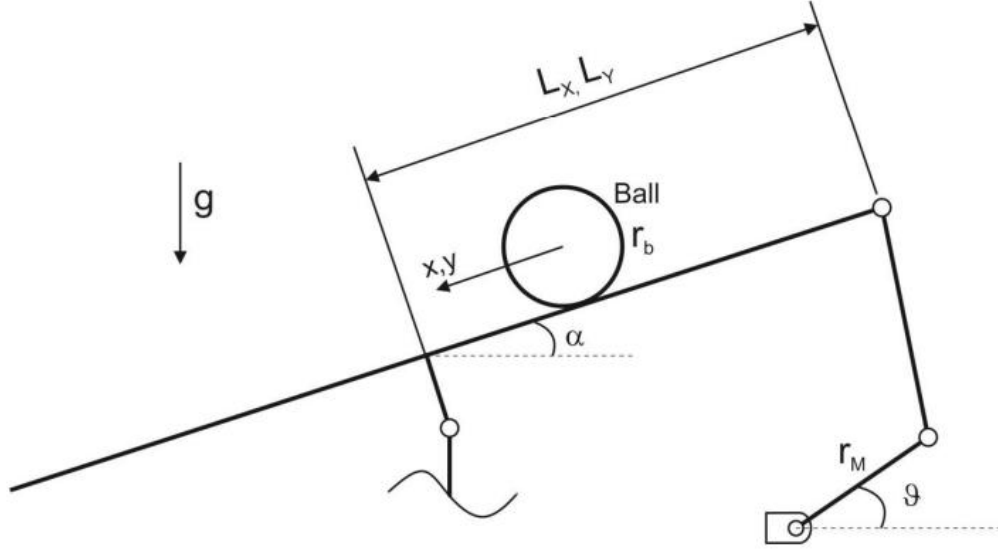


Figure 7. Free body diagram

2.1.3 Equations of motion

To apply the Newton's Law of Motion, the system should be separated into two parts (x and y direction). Considering the motion in x-direction, according to the Newton's Law of Motion, the summation of forces acting on the ball equals:

$$\sum F = m_b \frac{d^2 x_b}{dt^2} \quad (2.1)$$

Since there is no friction and viscous damping in the system (part of our assumptions in previous section, 2.1.2) the forces acting on the ball can be described as:

$$\sum F = F_{x,t} - F_{x,r} \quad (2.2)$$

Where $F_{x,t}$ is the translational force acting on the ball due to gravity and inclined plane (shown in Figure 7.).

$$\sum F_{x,t} = m_b g \sin(\alpha) \quad (2.3)$$

$F_{x,r}$ is the force acting on the ball due to the ball's rotation. The torque generated due to the ball's rotation is:

$$T = F_{x,r} * r_b = \frac{J_b}{r_b} \ddot{x}_b \longrightarrow F_{x,r} = \frac{J_b}{r_b^2} \ddot{x}_b \quad (2.4)$$

Thus;

$$m_b \ddot{x}_b = m_b g \sin(\alpha) - \frac{J_b}{r_b^2} \ddot{x}_b \quad (2.5)$$

By solving the equation for $\ddot{x}_b$, we get the required form of the equation:

$$\ddot{x}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \sin(\alpha) \quad (2.6)$$

Since all steps can be applied for y direction, $\ddot{y}_b$, is also found:

$$\ddot{y}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \sin(\beta) \quad (2.7)$$

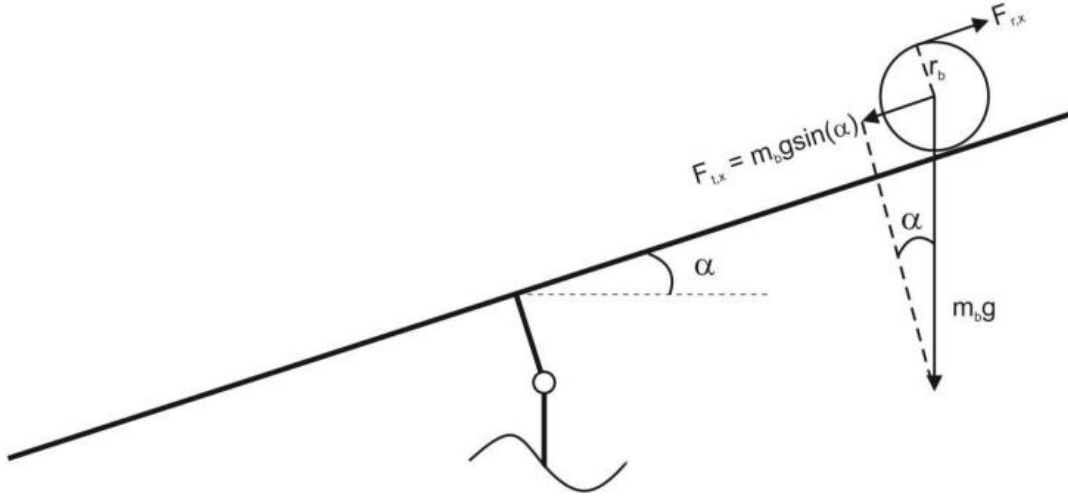


Figure 8. Relationship between motor angles and plate angles

The purpose of modeling is to derive the physical relationship between the outputs and inputs of a system. For our case, ball balance table, the outputs of the system are the coordinates of the ball in x and y-direction (x_b , y_b). The inputs of the system are the motor angles (θ_x , θ_y).

Figure 8. shows that:

$$\sin(\theta_x) r_m = \sin(\alpha) L_x = h \quad (2.8)$$

Thus, the following relationship between plate angle and motor angle can be obtained:

$$\sin(\alpha) = \frac{r_m}{L_x} \sin(\theta_x) \quad (2.9)$$

Same can applied to y direction (similar to 2.7):

$$\sin(\beta) = \frac{r_m}{L_y} \sin(\theta_y) \quad (2.10)$$

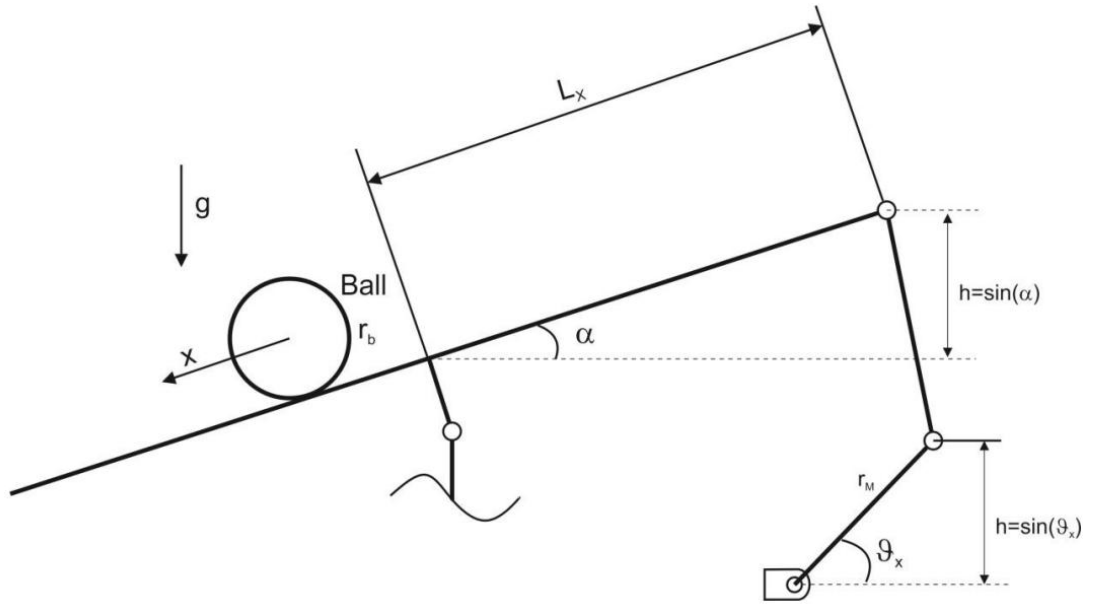


Figure 9. Relationship between motor angles and plate angles

So, when we put equivalents of $\sin(\alpha)$ and $\sin(\beta)$ found in 2.9 and 2.10 to replace in the ones in the equations 2.6 and 2.7, we get the differential equations of the system which describe the relationship between the outputs and inputs of the system:

$$\ddot{x}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \frac{r_m}{L_x} \sin(\theta_x) \quad (2.11)$$

$$\ddot{y}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \frac{r_m}{L_y} \sin(\theta_y) \quad (2.12)$$

2.1.4 Linearization the system

To find the transfer function of the system, the differential equations are linearized around the operating point at $x=0$ and $y=0$. For smaller angles:

$$\sin(\theta_x) = \theta_x, \sin(\theta_y) = \theta_y \quad (2.13)$$

Thus Equations 2.11 and 2.12 can be re-written as:

$$\ddot{x}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \frac{r_m}{L_x} \theta_x = K_{bbt,x} \theta_x \quad (2.14)$$

$$\ddot{y}_b = \frac{m_b g r_b^2}{m_b r_b^2 + J_b} \frac{r_m}{L_y} \theta_y = K_{bbt,y} \theta_y \quad (2.15)$$

By using the value of system parameters listed in Table 1, $K_{bbt,x}$ and $K_{bbt,y}$ is calculated as 1 and 1.255 respectively.

2.1.5 Obtaining the transfer function of plant

By taking the laplace transformation of 2.14 and 2.15 we get:

$$s^2 X_b(s) = K_{bbt,x} \theta_x(s) \quad (2.16)$$

$$G_{bbt,x}(s) = \frac{X_b}{\theta_x} = \frac{K_{bbt,x}}{s^2} = \frac{1}{s^2} \quad (2.17)$$

Again, by doing the same for y axis, we get:

$$G_{bbt,y}(s) = \frac{X_b}{\theta_y} = \frac{K_{bbt,y}}{s^2} = \frac{1.255}{s^2} \quad (2.18)$$

2.1.6 Modeling of actuator

The dynamic behavior of the servomotor can be approximated by a first-order transfer function, expressed in the following form:

$$G_m = \frac{\theta(s)}{V_m(s)} = \frac{K_m}{\tau s + 1}$$

K_m is the gain and τ is the time constant of the actuator. The values of them is not calculated by us but given by the producer. $K_m=100$ and $\tau = 0.01$

2.1.7 Simplified block diagram of operation

After modeling actuator and plant there is one thing left to operate the system, adding the controller. The schematic diagram of operation is shown below, in figure 9.

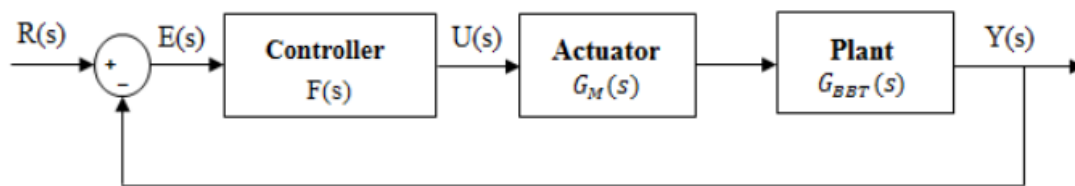


Figure 10. Control schematic of operation

2.2. PID Tuning by Gradient Descent Algorithm

2.2.1 Gradient descent

Gradient descent is (as explained by Ruder (2016)) an optimization algorithm used to minimize an objective function, which is dependent on a model's parameters. It works by adjusting the parameters in the direction opposite to the gradient of the objective function with respect to those parameters. The learning rate η controls the size of the steps taken toward the (local) minimum. Essentially, the algorithm moves along the slope of the surface defined by the objective function, gradually descending towards a valley. The mathematical formula of iteration of gradient descent can be expressed as follows:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} f(\theta_t) \quad (2.19)$$

Where θ_t is the parameter vector at iteration t , α is the step size, $\alpha \nabla_{\theta} f(\theta_t)$ is the gradient of the objective function f with respect to θ evaluated at θ_t , and θ_{t+1} is the updated parameter vector. The operation (or algorithm) can be visualised as shown in figure 10.

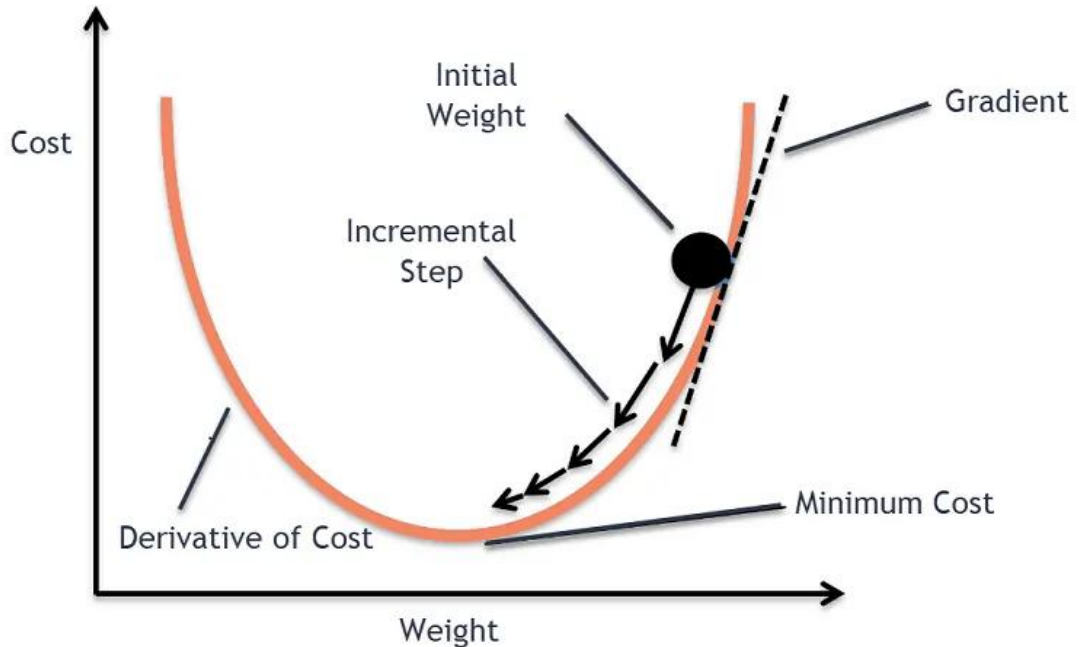


Figure 11 Operation of Gradient Descent Algorithm

Even though the standard gradient descent algorithm is simple and easy to apply, it has some limitations that can affect its performance in practice. One of the biggest problems is picking the step size α . The step size has a huge effect on how quickly the

algorithm converges and where it goes. If the step size is too small, the algorithm might take forever to converge, but if it's too large, it could fail to converge or even cause the algorithm to diverge.

Another issue stated by Hajji, Benhala and Hamdi (2024), gradient descent can be sensitive to the scale and conditioning of the objective function, which can make it converge slowly or cause it to bounce around the optimal solution. Another issue is that gradient descent can't escape local optima because it always moves in the steepest descent direction, ignoring the curvature of the objective function. This means that the standard gradient descent algorithm can take a long time to converge, especially in high-dimensional spaces or when dealing with complex optimization problems.

2.2.2 Implementation of gradient descent

To implement this algorithm to our system, first we need to decide what is our objective function. In control systems, the aim is the reducing the error and make the system reach desired input as fast as possible. So our cost function, (objective function to be found its local minima) should contain both error and time. We will be using time-weighted absolute error integral for this reason.

Time weighted absolute error integral is a one of the performance metrics used in control systems. It evaluates how well a system responds to a setpoint or desired value over time. It gives more weight to errors that occur later in time, penalizing long-lasting errors more than early ones since the it scales directly proportional with time. That means we are expecting lower settling time at the cost of high peak value of error. The formula of it given below:

$$\text{Time weighted absolute error} = \int_0^T t * |e(t)| dt \quad (2.20)$$

After that the only thing needed to be decided is step size. It is widely recommended that to use 0.02 or 0.05 for this method. However, instead of using one of these we used 4 different step sizes, 0.02, 0.05, 0.1, 0.9 for sake of creating samples to observe its effect. Theoretically, with enough iteration it is possible to get near-same outcomes since step size only affects how fast the algorithm learns.

While applying gradient descent method to PID controllers, there is one special problem occurs. Because of the nature of PID, gains can not be nonpositive. If at the any iteration of operation pid gains become nonpositive, we will not be able to iterate further, since simulating a PID controller will be impossible, there will be no error to calculate and progress. To ensure positive PID gains (which also means ensuring unconstraint of problem), we optimize in the logarithmic domain. Instead of optimizing the gains K_p , K_i , K_d directly, we define new variables:

$$\theta_p = \log K_p , \theta_i = \log K_i , \theta_d = \log K_d \quad (2.21)$$

This transformation allows us to perform unconstrained optimization over $\theta_p, \theta_i, \theta_d$ while ensuring that the actual gains recovered via the exponential function $K = e^\theta$ are always positive.

Most of tuning process and progress comparison is made on matlab workspace with the code in Appendix A, since Simulink does not support iterative optimisation techniques. However, the effectiveness of end results are tested in simulink program, shown in Appendix B.

3. RESULTS

3.1. Tuned PID Controller Simulation

3.1.1 Step responses with different step sizes

As discussed in section 2.2.2, tuning is done with 4 different step sizes. In this case, the number of iterations were 50000 in the beginning. But with that much iteration it was nearly impossible to see the difference between the responses acquired by using PID gains obtained by each rate. For better visualization and observation step responses at 5000 iteration of all step sizes are given in fig 11.

As expected, with the increase of step size, system is closer desired optimal solution. Again as discussed in section 2.2.2, the aim of this algorithm was reach the steady state faster because of our cost function. Remember our cost function increases higher with errors at later times. So, lower cost means faster reach to steady state.

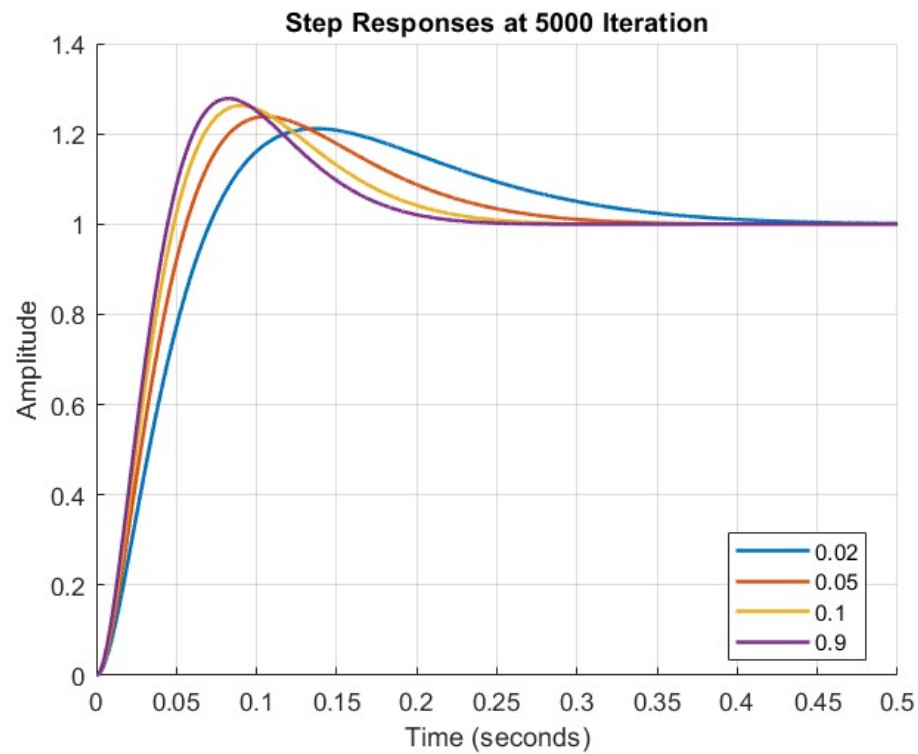


Figure 12.X-axis step responses of different step sizes at 5000 Iteration

However, at 20000 iteration, it is nearly imposible to see the difference between responses (shown in figure 12).

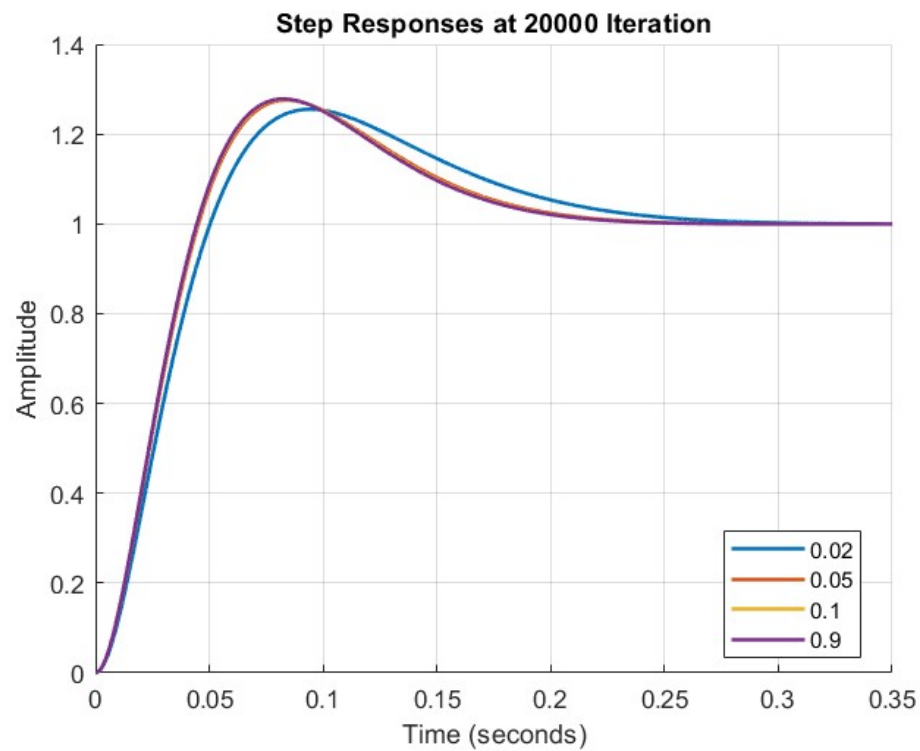


Figure 13. X-axis step responses of different step sizes at 20000 iteration

To show a more precise information about peak values, rise times and settling times at 5000 iteration and 20000 iteration is given in table 2 and table 3 respectively.

Table 2 Peak value, rise time and settling time at 5000 iteration for different step sizes

<i>Step Size</i>	<i>Peak value</i>	<i>Rise time (s)</i>	<i>Settling time (s)</i>
0.02	1.21	0.1426	0.432
0.05	1.24	0.1033	0.329
0.1	1.26	0.0924	0.269
0.9	1.28	0.0837	0.238

Table 3. Peak value, rise time and settling time at 20000 iteration for different step sizes

<i>Step Size</i>	<i>Peak value</i>	<i>Rise time (s)</i>	<i>Settling time (s)</i>
0.02	1.25	0.1013	0.278
0.05	1.28	0.0822	0.240
0.1	1.28	0.0813	0.238
0.9	1.28	0.0792	0.237

3.1.2 Cost progression with different step sizes

In addition to examining the step responses, we also looked at how the cost function changes over time for different step sizes. This helps to understand how efficiently each step size improves the system performance during the tuning process.

Again, as explained earlier in Section 2.2.2, the cost function is designed to penalize errors more heavily at later time steps. So, a lower overall cost indicates that the system reaches steady state faster. By tracking how the cost decreases throughout the iterations, it becomes easier to compare how effective each step size is during optimization.

Figure 13 shows how the cost evolves for step sizes of 0.02, 0.05, 0.1, and 0.9. As expected, higher step sizes reduce the cost more quickly.

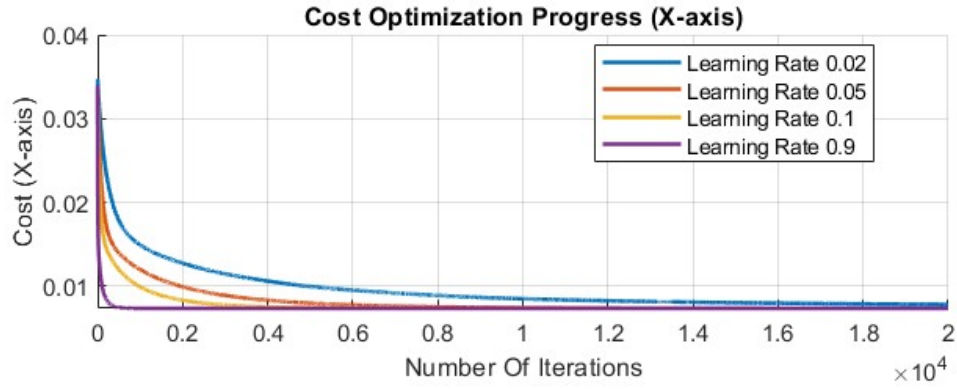


Figure 14. X-axis cost vs. number of iterations for different step sizes

As it seen from figure 13, with enough iterations, the cost is near-same for all step sizes. The X-axis cost for all step sizes at 5000 iteration and 20000 iteration is shown below in table 4 and 5 respectively.

Table 4. X-axis cost at 5000 iterations for different step sizes

<i>Step Size</i>	<i>Cost</i>
0.02	0.00995708
0.05	0.00796533
0.1	0.00742253
0.9	0.00734097

Table 5. X-axis cost at 20000 iterations for different step sizes

<i>Step Size</i>	<i>Cost</i>
0.02	0.00781605
0.05	0.00734405
0.1	0.00734158
0.9	0.00734034

As shown in all tables (table 2,3,4,5) change in result is infinitesimal (after a while).

3.1.3 Effect of step size on iteration time

For research purposes, the duration of each iteration was recorded across all step sizes. The effect of step size on iteration time was found to be minimal. In other words, increasing or decreasing the step size had little to no measurable impact on how long each iteration took to complete during the gradient descent process.

This outcome is expected because the iteration time is primarily determined by the hardware on which the algorithm runs, which remains constant (assuming no other significant processes are running on the device), regardless of the step size. Since the step size only affects the magnitude of parameter updates and not the size or duration of the evaluation window, the time taken for each iteration stayed approximately the same.

Therefore, while the step size plays a significant role in convergence speed and the final controller performance, its influence on the iteration time itself is negligible.

For further information, the total time for 20,000 iterations and the average iteration time for different step sizes are presented in Table 6 below (X-axis).

Table 6. Effect of Step sizes on Iteration Time

<i>Step Size</i>	<i>Total Time (ms)</i>	<i>Avg Time per Iteration</i>
<i>0.02</i>	650.51	32.5253
<i>0.05</i>	654.10	32.7049
<i>0.1</i>	633.53	31.6767
<i>0.9</i>	668.71	33.4357

3.1.4 Final results on workspace and simulink

In this part, PID gains collected by gradient descent algorithm (step size 0.9, 20000 iteration) will be applied to both simulink diagram in Appendix B and matlab's `step()` function. Applied PID gains K_p , K_i and K_d values are 4.655, 0.0851 and 0.3570 for X-axis; 4.1839, 0.0860 and 0.3016 respectively. Blue line shows X position and orange line shows Y position in all figures.

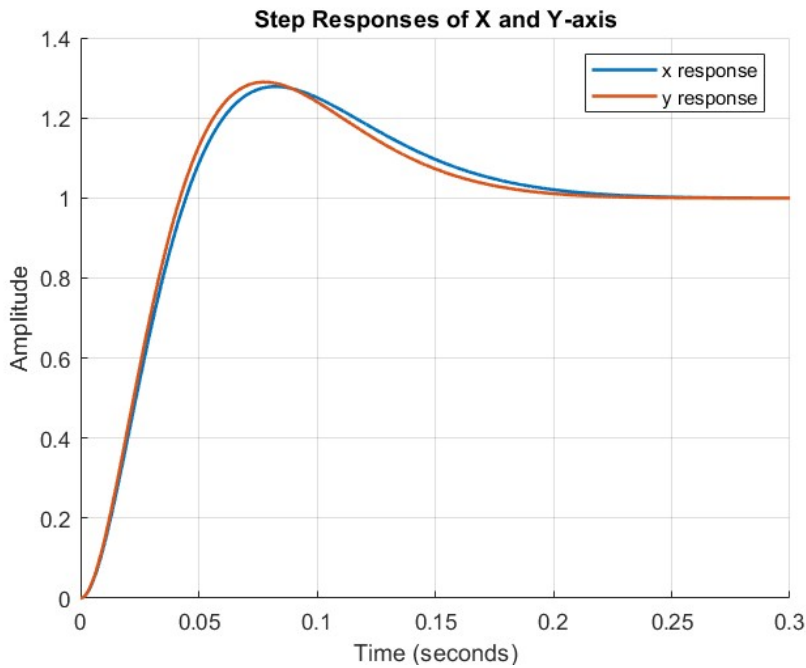


Figure 15. Step responses of X and Y-axis in 2D (Workspace)

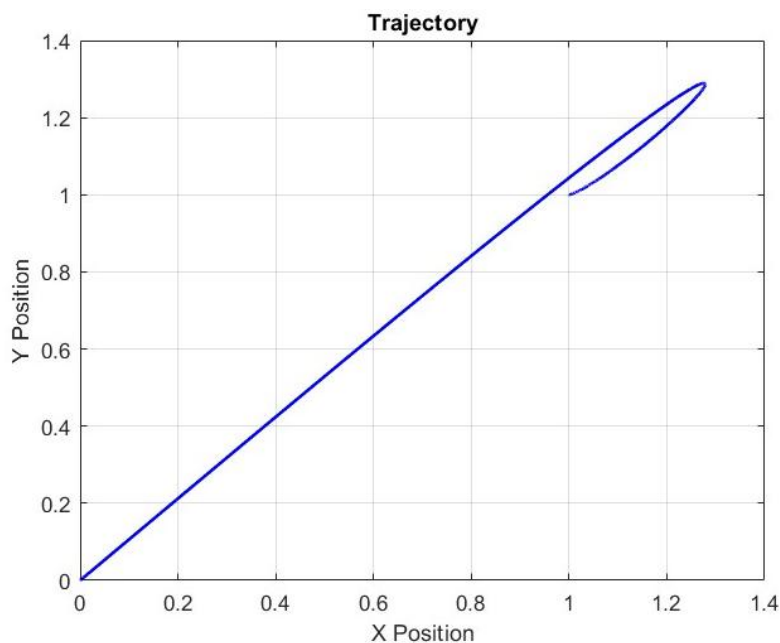


Figure 16 Step response of system in 3D (Workspace)

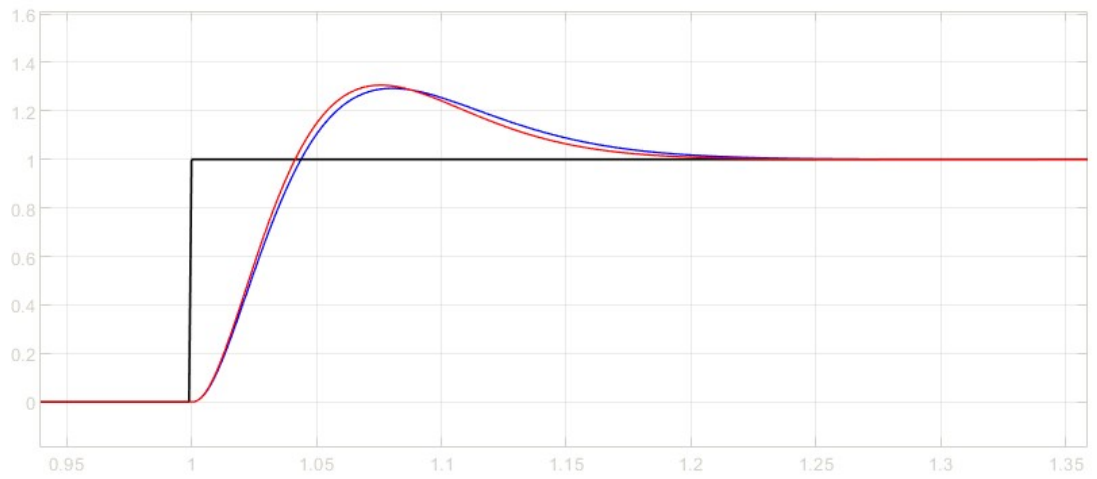


Figure 17. Step response in Simulink

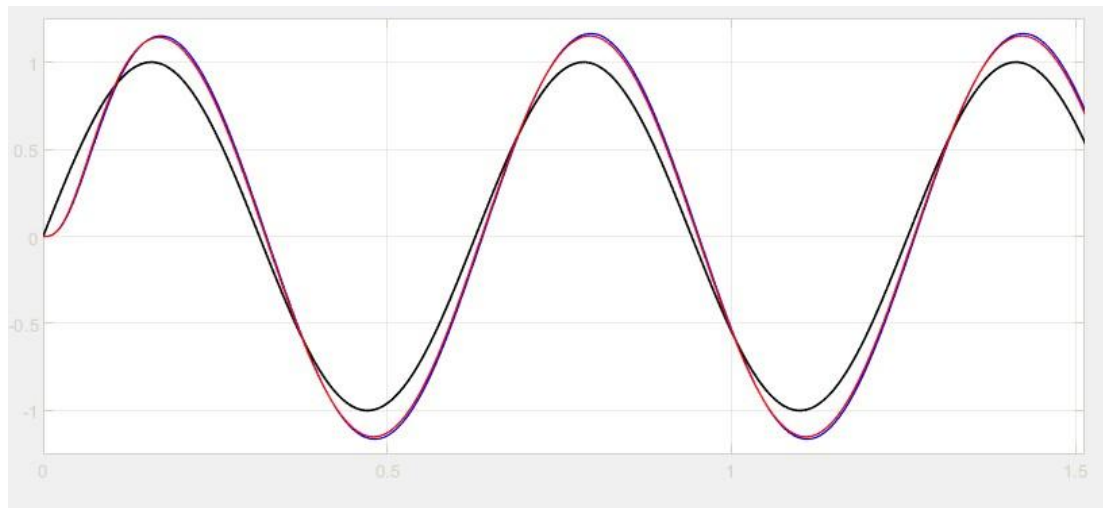


Figure 18. System's response to sinusoidal input in simulink



Figure 19. System's response to square input in simulink



Figure 20. System's response to sawtooth input in simulink

When applying square and sawtooth inputs, the overshoots after the first input are larger than the initial one. For the square input, this is because the system first moves the ball from 0 to -1, a distance of magnitude 1. However, on the second transition, it must move the ball from -1 to 1, a distance of magnitude 2. Covering a greater distance in the same amount of time leads to a larger overshoot.

In the case of the sawtooth input, the ball is already moving in one direction and then suddenly receives an input in the opposite direction. This abrupt reversal requires the system to counteract the ball's momentum, which results in greater overshoot.

3.2. LQI

We can control the Ball balancing table with design a LQI controller. As you remember, we can begin by obtaining the state space matrix of the system. We are using the transfer function to obtain the matrix which is used in the PID controller.

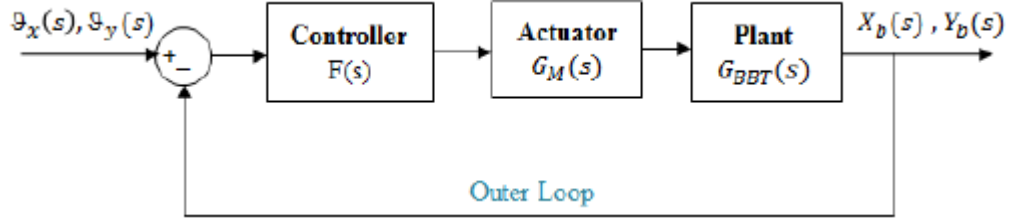


Figure 21: The block diagram of the system.

The transfer function of the Actuator for x-axis and y-axis is the same:

$$G_M(s) = \frac{100}{0.01s + 1} \quad (3.1)$$

The transfer function of the plant.

$$\text{X-axis:} \quad G_{BBT,X}(s) = \frac{1}{s^2} \quad (3.2)$$

$$\text{Y-axis:} \quad G_{BBT,Y}(s) = \frac{1.255}{s^2} \quad (3.3)$$

We are going to find two state space matrixes, then we will combine them. Let's start with x-axis.

$$H(s) = \frac{Y(s)}{U(s)} = \frac{\frac{100}{(0.01s + 1)s^2}}{1 + \frac{100}{(0.01s + 1)s^2}} = \frac{100}{0.01s^3 + s^2 + 100}$$

$$100 U(s) = 0.01s^3 Y(s) + s^2 Y(s) + 100 Y(s) \quad (3.4)$$

$$x(t) = y(t)$$

$$x_2(t) = \dot{y}(t) = \dot{x}(t) \quad (\text{Location})$$

$$x_3(t) = \ddot{y}(t) = \ddot{x}(t) \quad (\text{Speed})$$

$$\dot{x}_3(t) = \dddot{y}(t) \quad (\text{Acceleration})$$

$$\ddot{y}(t) = 100 * 10^2 u(t) - 100 \dot{y}(t) - 100 * 10^2 y(t) \quad (3.5)$$

Let's write the state space matrix.

The state space matrix of x-axis.

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -10000 & 0 & -100 \end{bmatrix}_{3 \times 3} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{3 \times 1} + \begin{bmatrix} 0 \\ 0 \\ 10000 \end{bmatrix}_{3 \times 1} u(t) \quad (3.6)$$

$$y(t) = [1 \ 0 \ 0]_{1 \times 3} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{3 \times 1} + [0] u(t) \quad (3.7)$$

$$A_x = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -10000 & 0 & -100 \end{bmatrix}_{3 \times 3}, B_x = \begin{bmatrix} 0 \\ 0 \\ 10000 \end{bmatrix}_{3 \times 1}, C_x = [1 \ 0 \ 0]_{1 \times 3} \quad (3.8)$$

$$D_x = [0]$$

We can obtain the state-space matrixes for the y-axis while following the same way which is used for x-axis.

$$A_y = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -12550 & 0 & -100 \end{bmatrix}_{3 \times 3}, B_y = \begin{bmatrix} 0 \\ 0 \\ 12550 \end{bmatrix}_{3 \times 1}, C_y = [1 \ 0 \ 0]_{1 \times 3} \quad (3.9)$$

$$D_y = [0]$$

Our system has two inputs and two outputs, so we can control them with one LQI controller due to multi-input multi-output (MIMO). In order to do that we should combine the matrixes.

$$A = \begin{bmatrix} A_x & 0 \\ 0 & A_y \end{bmatrix}_{6 \times 6}, B = \begin{bmatrix} B_x & 0 \\ 0 & B_y \end{bmatrix}_{6 \times 2}, C = \begin{bmatrix} C_x & 0 \\ 0 & C_y \end{bmatrix}_{2 \times 6} \quad (4)$$

$$D = \begin{bmatrix} D_x & 0 \\ 0 & D_y \end{bmatrix}_{2 \times 2}$$

Let's see the LQR controller we implemented to the MATLAB. You can see the code on page the Appendix C, LQR Control MATLAB Code_1.

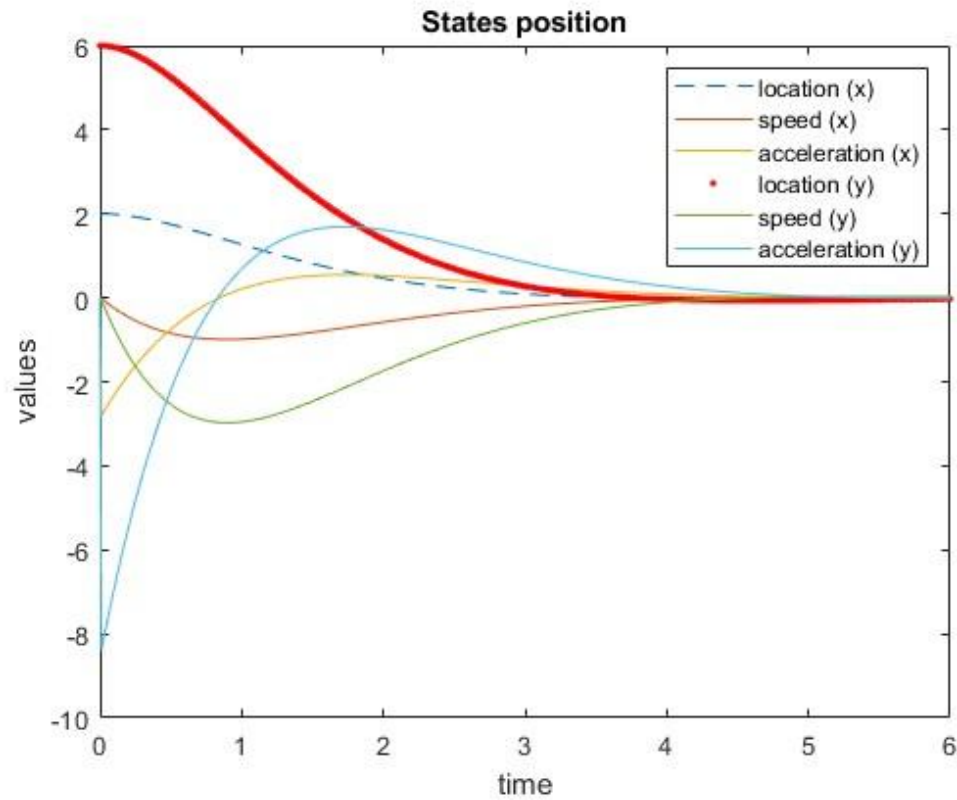


Figure 22: The graph of the LQR states.

You can see six lines on the graph, representing different location points. For the x-axis, the location is set to 2, and for the y-axis, it is set to 6. The speed and acceleration of the y-axis states are higher than those of the x-axis states because the y-position is greater. As a result, all states converge to zero in approximately 5 seconds.

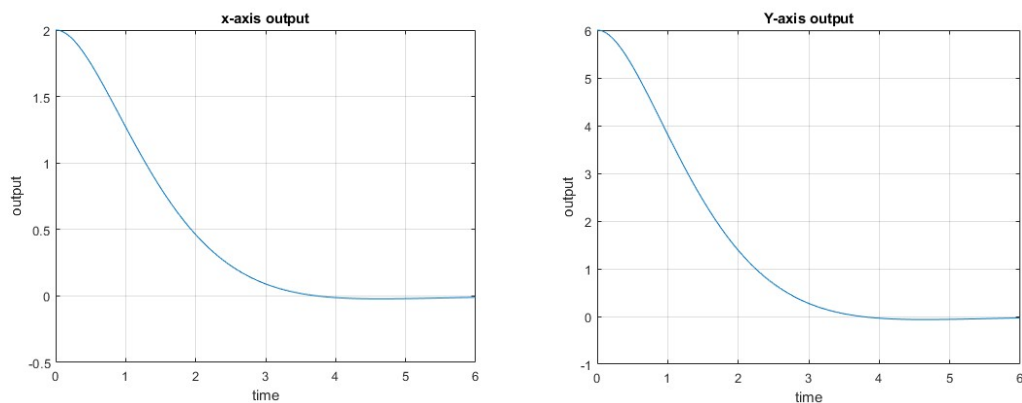


Figure 23: The output of the x-axis, and the y-axis.

As we can see from the graphs, the LQR controller drives the system states to zero without requiring a reference input. To use an LQI controller, we need to add an integrator to the system.

$$\begin{aligned} \text{Discrete time state-space: } x(k+1) &= Ax(k) + Bu(k) \\ y(k) &= Cx(k) + Du(k) \end{aligned} \quad (4.1)$$

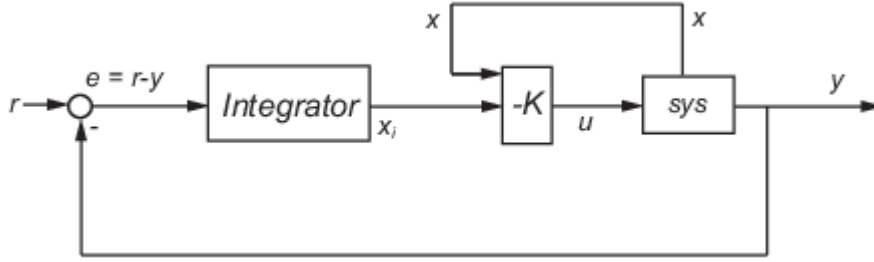


Figure 24: The block diagram of LQI

In the LQR controller, we have the control input $u = -Kx$. This formulation will change when we add integral action. The integrator accumulates the error over time and generates a corrective signal to eliminate steady-state error. First, let's examine the cost function. The state feedback control is updated as $u = -[K_x \ K_i] \begin{bmatrix} x \\ x_i \end{bmatrix}$.

Where, K_x represents the state feedback gain vector, K_i represents the integral gain, and x_i is the additional integral state. The state vector becomes augmented due to integration. For example, if the original system has 3 states (position, velocity, and acceleration), adding the integral state x_i extends it to 4 states. Consequently, the gain vector will contain 4 elements, with the last one being the integral gain K_i .

The gains define according to Q and R , they are cost weight of state variable, cost of control signal respectively. Then the gain determines by solving Riccati equation.

$$\text{The cost function: } J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k) \quad (4.2)$$

This is the cost of function according to discrete time. While the Q focuses on states, and the R focuses on the control signal. To obtain the gains we get helps of Riccati equation.

$$P = \tilde{A}_d^T P \tilde{A}_d - \tilde{A}_d^T P \tilde{B}_d (R + \tilde{B}_d^T P \tilde{B}_d)^{-1} \tilde{B}_d^T P \tilde{A}_d + Q \quad (4.3)$$

This is the Riccati equation (4.3). The Riccati equation estimates the next cost. It will solve again up to convergence to $P_{k+1} \approx P_k$.

The gain equation:
$$K = (R + \tilde{B}_d^T P \tilde{B}_d)^{-1} \tilde{B}_d^T P \tilde{A}_d \quad (4.4)$$

So, we converted the continuous time matrixes to the discrete time, now to use Riccati equation we need to extend the matrix A (system matrix) and the matrix B (input matrix).

$$\epsilon(k+1) = \epsilon(k) + Ts[r(k) - \epsilon(k)] \quad (4.5)$$

When we combine the error equation (4.5) with discrete time state space (4.1) we can obtain the below augmented plant.

$$\begin{bmatrix} x(k+1) \\ \epsilon(k+1) \end{bmatrix} = \begin{bmatrix} A & 0 \\ -C * Ts & I_m \end{bmatrix} \begin{bmatrix} x(k) \\ \epsilon(k) \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u(k) + \begin{bmatrix} 0 \\ I_m \end{bmatrix} r(k) \quad (4.6)$$

Therefore, we can obtain extended matrixes as below to use in Riccati equation.

$$A_{ext} = \begin{bmatrix} A & 0 \\ -C * Ts & I_m \end{bmatrix}, \quad B_{ext} = \begin{bmatrix} B \\ 0 \end{bmatrix} \quad (4.7)$$

Let's implement them on MATLAB. You can see the code on page Appendix D, LQI Control MATLAB Code_2. The LQI control MATLAB code_2 is continuous of LQR control MATLAB code_1.

A_ext:

0.9987	0.01	0	0	0	0	0	0
-0.3678	0.998	0.0063	0	0	0	0	0
-63.1836	-0.3678	0.3668	0	0	0	0	0
0	0	0	0.9983	0.01	0	0	0
0	0	0	-0.4616	0.9983	0.0063	0	0
0	0	0	-79.2863	-0.4616	0.3666	0	0
-0.01	0	0	0	0	0	1	0
0	0	0	-0.01	0	0	0	1

B_ext:

0.0013	0
0.3678	0
63.1836	0
0	0.0017
0	0.4616
0	79.2863
0	0
0	0

KK (gain):

-	0.09311	0.00621	-	-	-	-	2.3170
0.7092	035909	444098	2.5185	8.02603	3.32299	0.3067	561061
784129	89379	561396	063882	613194	698000	239931	9129e-
19314			9713e-	852e-15	812e-17	62770	14
-	-	-	-	0.07329	0.00495	4.6413	-
3.1428	6.69843	2.76080	0.7683	214684	051226	701036	0.2444
409909	247789	160822	270115	68654	332715	0093e-	157919
5241e-	226e-15	844e-17	18436			14	93277
14							

Generally, we obtain the state space of the modeling, then converted to discrete time because of our system controlling by Arduino and MATLAB. After the LQR controller added an integrator, we extended the matrixes to generate the gains, so let's look at the graphs.

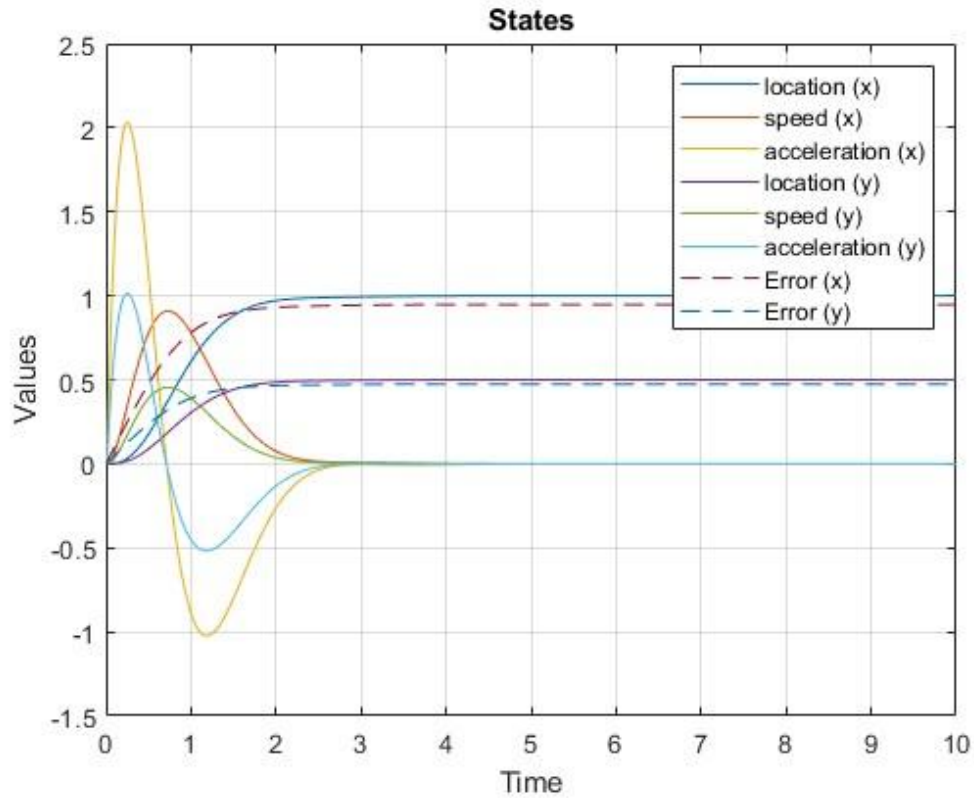


Figure 25: The output of LQI control with all states.

The system is set to references almost in 3 seconds. When the system set reference points, the other states as speed and acceleration become zero.

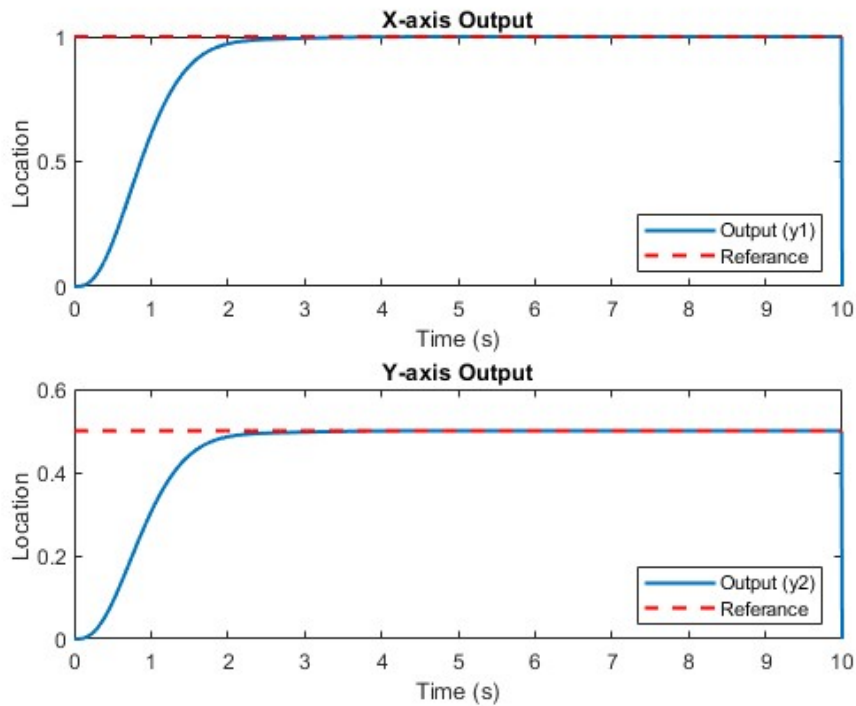


Figure 26: The output of x-axis and y-axis of the LQI.

If we increase the R , it means that the controller will focus on the cost more than the performance of the system, let's see it. Check the code in Appendix E.

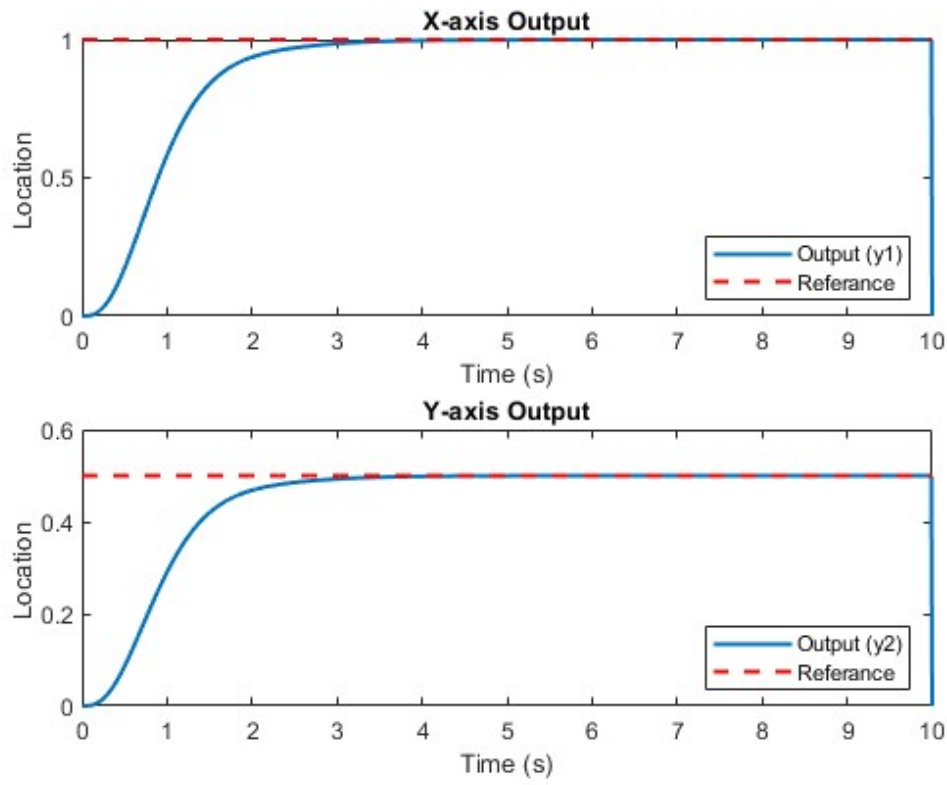


Figure 27: The new output with ignoring the performance.

As you see, the system set to references later according to previous control. On other hand, we can ignore the cost of the energy utilized as increase the Q matrix. See the code in Appendix E.

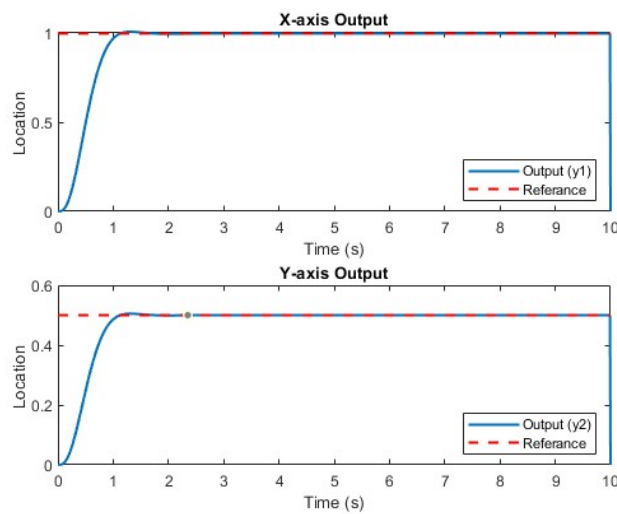


Figure 28: The new output with ignoring the cost

4. REFERENCES

- [1] Yao Wang & Xiaoli Li & Yang Li & Baoyong Zhao (2012). Identification of ball and plate system using multiple neural network models. International Conference on System Science and Engineering
- [2] D. Núñez & G. Acosta & J. Jiménez (2017) Control of a ball-and-plate system using a State-feedback controller. Revista Chilena de Ingeniería 28 (1), 6-15
- [3] Sebastian Ruder (2016) An overview of gradient descent optimization algorithms. arXiv.org
- [4] Morad Hajji & Bachir Benhala & Imad Hamdi (2024). 2024 4th International Conference on Innovative Research in Applied Science, Engineering and Technology
- [5] Gradient Descent Algorithm: How Does it Work in Machine Learning? (2025) <https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/> Accessed 1 May 2025
- [6] Nikunj Mehta & Dharmendra Chauhan & Sagarkumar B. Patel & Siddharth Mistry (2017) Design of HMI Based on PID Control of Temperature. International Journal of Engineering Research & Technology
- [7] How Resistive Touch Screen Technology Works? (2025) <https://www.reshine-display.com/how-resistive-touch-screen-technology-works.html/> Accessed 2 May 2025
- [8] Abraham, R. J. (2024). A brief review of the development and evolution of optimal control strategies. Retrieved May 8, 2025, from <https://www.sciencedirect.com/science/article/pii/S2666720724000171>
- [9] Parlar, İ. (2025, April 19). Linear Quadratic Regulator. ElektrikPort. Retrieved April 19, 2025, from [https://www.elektrikport.com/universite/lqr-\(linear-quadratic-regulator\)-nedir/23347#ad-image-0](https://www.elektrikport.com/universite/lqr-(linear-quadratic-regulator)-nedir/23347#ad-image-0)
- [10] Feng, Z., & Zhu, J. (2007). Design of continuous and discrete LQI control systems with stable inner loops. Journal of Shanghai Jiaotong University (Science), 12(6), 787–792.
- [11] Zarzycki, K., & Ławryńczuk, M. (2021). Fast real-time model predictive control for a ball-on-plate process. Sensors, 21(12), 3959. <https://doi.org/10.3390/s21123959>

[12] MathWorks. (2025). Linear-Quadratic-Integral control [Computer software documentation]. Retrieved April 18, 2025,

from <https://www.mathworks.com/help/control/ref/ss.lqi.html>

[13] Kasnakoğlu, C. (2014). Ayırık-zamanlı kontrol (~ dijital kontrol) [Lecture notes]. Wordpress. Retrieved May 3, 2025,

https://kasnakoglu.wordpress.com/wp-content/uploads/2014/01/ders9_ck01a.pdf

Appendix A
Core Gradient Descent PID Tuning Code

```
function [Kp_opt, Ki_opt, Kd_opt, cost_history, time_history] =...
    optimize_axis(Gplant, Gm, initial_guess, learning_rate, max_iter, axis_name)
    Kp_uncon = log(initial_guess(1));
    Ki_uncon = log(initial_guess(2));
    Kd_uncon = log(initial_guess(3));

    cost_history = zeros(max_iter, 1);
    time_history = zeros(max_iter, 1);
    epsilon = 0.0001;

    for iter = 1:max_iter
        tic; %start timer for computations

        Kp = exp(Kp_uncon);
        Ki = exp(Ki_uncon);
        Kd = exp(Kd_uncon);

        current_cost = compute_cost(Kp, Ki, Kd, Gplant, Gm);
        cost_history(iter) = current_cost;

        % Finite differences for gradients
        cost_Kp = compute_cost(Kp + epsilon, Ki, Kd, Gplant, Gm);
        grad_Kp_con = (cost_Kp - current_cost) / epsilon;

        cost_Ki = compute_cost(Kp, Ki + epsilon, Kd, Gplant, Gm);
        grad_Ki_con = (cost_Ki - current_cost) / epsilon;

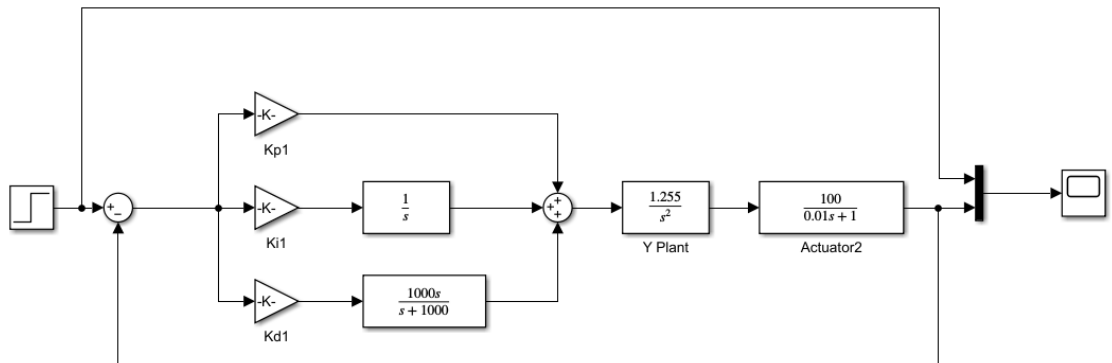
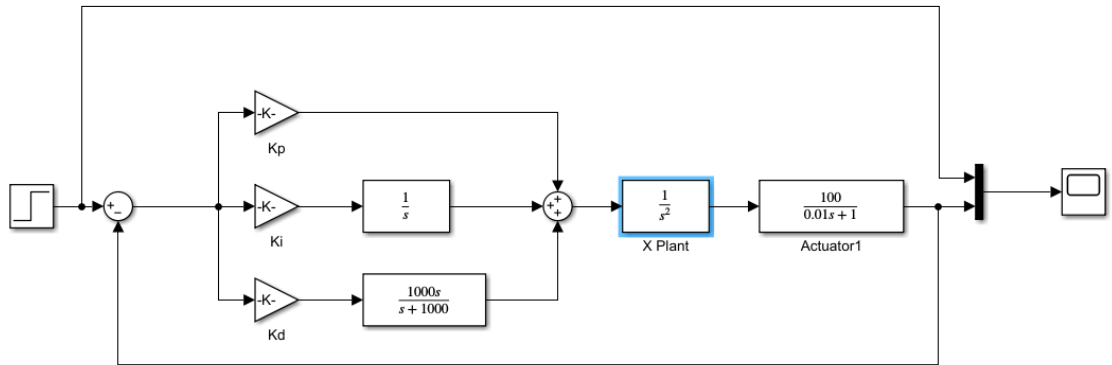
        cost_Kd = compute_cost(Kp, Ki, Kd + epsilon, Gplant, Gm);
        grad_Kd_con = (cost_Kd - current_cost) / epsilon;

        % Convert to unconstrained gradients
        grad_Kp_uncon = grad_Kp_con * Kp;
        grad_Ki_uncon = grad_Ki_con * Ki;
        grad_Kd_uncon = grad_Kd_con * Kd;

        % Update parameters
        Kp_uncon = Kp_uncon - learning_rate * grad_Kp_uncon;
        Ki_uncon = Ki_uncon - learning_rate * grad_Ki_uncon;
        Kd_uncon = Kd_uncon - learning_rate * grad_Kd_uncon;

        time_history(iter) = toc; %stop timer
```


Appendix B
PID Control Simulation of Ball Balance Table in Simulink



Appendix C
LQR Control MATLAB Code_1

```
A = [0 1 0 0 0 0; 0 0 1 0 0 0; -10000 0 -100 0 0 0; 0 0 0 0 1 0; 0 0 0 0 0 1; 0 0 0 -12550
0 -100];
B = [0 0; 0 0; 10000 0; 0 0; 0 0; 0 12550];
C = [1 0 0 0 0 0; 0 0 0 1 0 0];
D = [0 0;0 0];
ctime = ss(A,B,C,D);
Ts = 0.01;

% ayrık zaman ( discrete time)
dtime = c2d(ctime,Ts,'zoh');
Q = eye(6);
R = diag([1,1]);
K = lqr(dtime,Q,R);
dtime2 = ss(dtime.A-dtime.B*K,-dtime.B*K,dtime.C-dtime.D*K,-dtime.D*K,Ts);

t = 0:Ts:6;
[y,t,x] = initial(dtime2,[2;0;0;6;0;0],t);
figure(1);
plot(t,x(:,1),'--');% states
hold on;
plot(t,x(:,2)); plot(t,x(:,3)); plot(t,x(:,4),'r.');
plot(t,x(:,5));plot(t,x(:,6));
legend('location (x)','speed (x)','acceleration (x)','location (y)','speed (y)','acceleration
(y)');
title ( 'States position');
xlabel('time'); ylabel('values');
figure(2);
plot(t,y(:,1));% output_x-axis
title ( 'x-axis output');
xlabel('time'); ylabel('output');grid on;
figure(3);
plot(t,y(:,2));%output_y-axis
title ( 'Y-axis output');
xlabel('time'); ylabel('output'); grid on;
```


Appendix D
LQI Control MATLAB Code_2

```
Ad = dtime.A;
Bd = dtime.B;
Cd = dtime.C;
cd = [Cd,zeros(2,2)];
% extended system matrixes
A_ext = [Ad, zeros(6,2); -Cd*0.01, eye(2)];
B_ext = [Bd; zeros(2,2)];

Qx = diag([500 10 1 500 10 1]);      % State weight
Qz = diag([10000, 10000]);           % Integrate weight
Q = blkdiag(Qx, Qz);

R = 0.1*eye(2);      % Control weight

% Riccati equation (DARE)
P = idare(A_ext, B_ext, Q, R);
KK = inv(R+ B_ext'*P*B_ext)* B_ext'*P*A_ext;
disp('KK:');
disp(KK);
K_integrate = KK(:,7:8);
K_state     = [KK(:,1:6),ones(2,2)];
%%
t = 0:Ts:10;
r = [ones(size(t)); 0.5*ones(size(t))];      % the reference for x and y
u = zeros(2, length(t));  % The control signals
x = zeros(6, length(t));  % The state Vectors
z = zeros(2, length(t));  % The integrate errors
y = zeros(2, length(t));  % The outputs
%%
```

Appendix D
LQI Control MATLAB Code_2

```
for k = 1:length(t)-1
    y(:,k) = Cd * x(:,k);
    % To update the integrate errors
    z(:,k+1) = z(:,k) + (r(:,k) - y(:,k)) * Ts;

    % The control signal (LQI)
    u(:,k) = -KK * [x(:,k); z(:,k)];

    % To update the states
    x(:,k+1) = Ad * x(:,k) + Bd * u(:,k);
end
%%
figure(4);
subplot(2,1,1);
plot(t, y(1,:), 'LineWidth', 1.5); hold on;
plot(t, r(1,:), '--r', 'LineWidth', 1.5);
title('X-axis Output'); xlabel('Time (s)'); ylabel('Location ');
legend('Output (y1)', 'Reference', 'Location', 'southeast');

subplot(2,1,2);
plot(t, y(2,:), 'LineWidth', 1.5); hold on;
plot(t, r(2,:), '--r', 'LineWidth', 1.5);
title('Y-axis Output'); xlabel('Time (s)'); ylabel('Location ');
legend('Output (y2)', 'Reference', 'Location', 'southeast');
%%
figure(5);
plot(t, u(1,:), 'LineWidth', 1.5); hold on;
plot(t, u(2,:), 'LineWidth', 1.5);
title('Control signals '); xlabel('Time (s)'); ylabel('u');
legend('u1 (X)', 'u2 (Y)');
```

Appendix D
LQI Control MATLAB Code_2

```
%%  
figure(6);  
plot(t,x);  
title('States');  
xlabel('Time');  
ylabel('Values')  
hold on;  
plot(t,z,'--');  
grid on;  
legend('location (x)','speed (x)','acceleration (x)','location (y)','speed (y)','acceleration (y)',  
      'Error (x)','Error (y)');
```


Appendix E
LQI Control MATLAB Code_2.1

Ignore the performance:

```
Qx = diag([100 1 1 100 1 1]);      % State weight
Qz = diag([400, 400]);              % Integrate weight
Q = blkdiag(Qx, Qz);
```

```
R = 50*eye(2);                      % Control weight
```

Ignore the cost:

```
Qx = diag([500 10 1 500 10 1]);    % State weight
Qz = diag([10000, 10000]);          % Integrate weight
Q = blkdiag(Qx, Qz);
```

```
R = 0.1*eye(2);                    % Control weight
```
